

C语言与程序设计

The C Programming Language

第10章 文件的输入与输出

王多强

1097412466

dqwang@hust.edu.cn

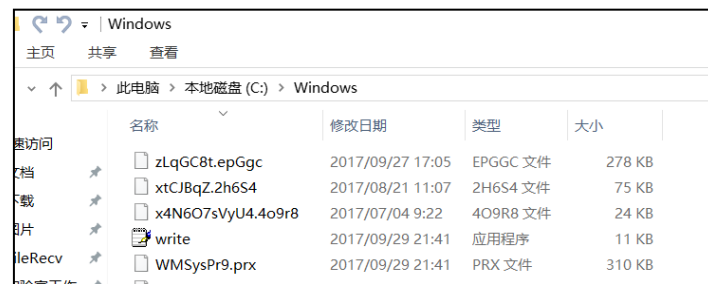
华中科技大学计算机学院

10.1 文件概述

文件是存放在**外存**中的**命名数据集合**。文件以“**文件名**”命名，保存在特定的位置，文件的位置以“**盘符+目录**”的方式进行索引，如

C:\、D:\myfile。

- 盘符：C:、D:、E: 等
- 目录：根目录及各级子目录
- 盘符及各级目录之间用 ‘\’ 隔开。



1. 文件的分类

文件按照**字节数据**的性质分为**文本文件**和**二进制文件**两大类。

1) 文本文件

以**字符型**数据构成的文件称为**文本文件**。

- ▣ **文本文件的内容组织：**

以**字节**为单位，由**字节序列**组成；字节中存放是**字符编码**（如字符的ASCII码，代表一个字符）。

- ▣ 文本文件可以用**文本编辑器**打开，**人可以直接阅读其内容**。

如：**.txt**、**.c**、**.h文件**等（命名不受任何限制，只以内容而定）

如：

序号	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
字节值	61	62	63	20	64	65	66	0D	0A	31	32	33	34	35	36	0D	0A
符号	a	b	c	空格	c	d	f	回车	换行	1	2	3	4	5	6	回车	换行

如果用文本编辑器打开文件，看到的内容是：

abc cdf

123456

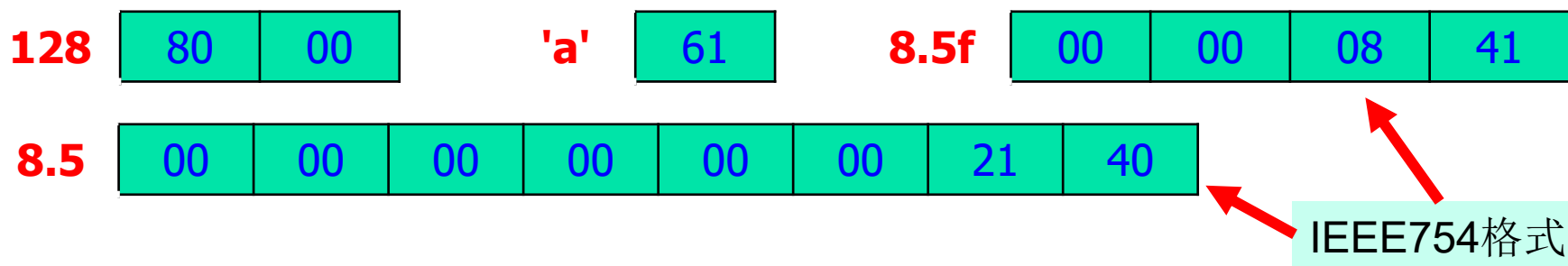
2) 二进制文件

二进制文件也是以**字节为单位**组织内容，也是**字节序列**，但字节的内容只被解释成**二进制 0-1 序列**，不再当作字符处理。

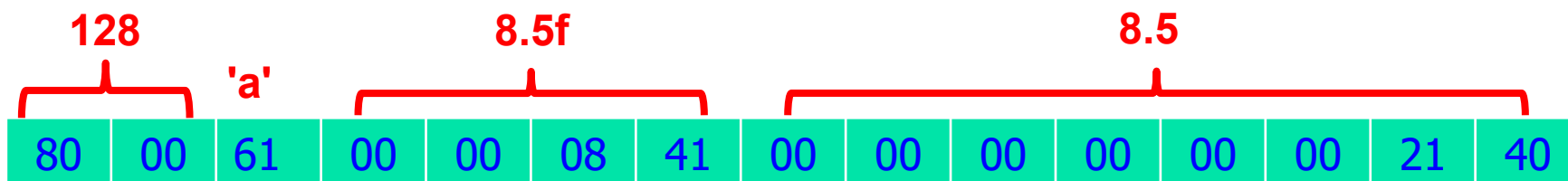
- ◆ **二进制文件一般是数据文件或可执行程序**，里面的字节内容是**数据数值的二进制表示或机器指令的二进制表示**。
 - 这类文件通常**不是给人看的**，而是**供程序使用的**。
 - 数据被写入二进制文件时，将按照其**数据类型**决定的**编码方式、数据长度**等直接存入文件，写入的字节内容与**数据在内存中的表示**完全一样——相当于将内存中数据的字节表示直接存入文件，所以二进制文件也被看作数据的**“内存映像”**。

例10.2 设 **128**, **'a'**, **8.5f**, **8.5** 四个数据依次为**短整型**、**字符型**、**单精度浮点型**和**双精度浮点型**。依次写入到一个二进制文件中。

◆ 四个数据各自的字节表示：



◆ 将它们依次写入一个二进制文件，**文件中字节顺序和内容**如下所示：



2、流式文件的概念

不管是文本文件还是二进制文件，**本质上都是由字节序列构成的**，所以文件可看作是一种“**字节流**”数据。而文本文件和二进制文件的区别只在于**如何解释字节内容**：是当作**字符**还是当作一般的**二进制数值**。

- ◆ 文本文件由表示字符的字节构成，所以它的“字节流”又称为**字符流**；而二进制文件就仅是一般的**二进制流**。
- ◆ **基于流的概念**，由字节序列构成的文件又称为**流式文件**：

从系统的角度，**流式文件没有内部结构**，仅被视为字节序列。
如何从逻辑上解释这些字节的内容将由应用程序负责。

3. 文件的操作

文件操作主要是**读**、**写**操作，文件的读写又称为**I/O操作**，即文件的输入（**Input**）、输出操作（**Output**）。

另外还有：**文件的创建**、**文件的删除**等

4、C语言对文件的输入与输出

C语言对文件的输入与输出有两种实现方法：

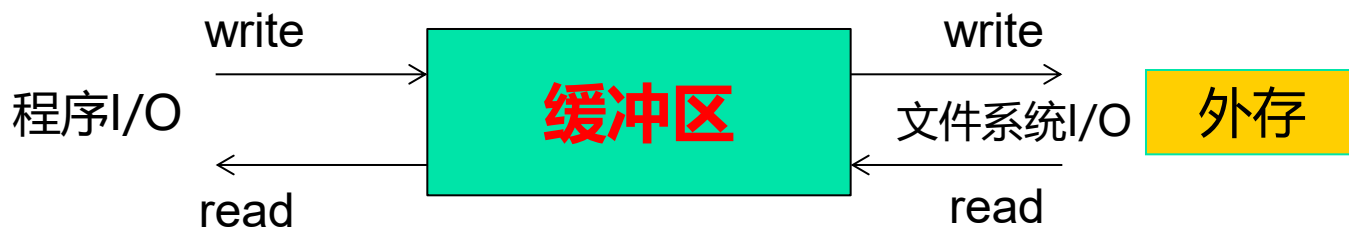
一种是通过**C语言自己的标准库函数**实现，称为**C的标准I/O**。

二种是借助**操作系统提供的底层I/O函数**实现，称为**底层I/O**。

1) C的标准I/O

C的标准I/O是一种基于**缓冲区**的输入输出。

缓冲区是程序开始运行时由**系统自动分配**的一块专门用于文件读写的**内存区域（连续字节空间）**，是连接程序与外设文件之间的桥梁。缓冲区在程序运行结束时由系统自动回收。



2) 底层I/O

底层I/O是由操作系统提供，由其**一组文件操作接口函数**实现。底层I/O的文件输入输出是**非缓冲形式**的。

5、标准 I/O 下的文件指针

读写文件时，系统要知道读写的是哪个文件（**文件名**）、这个文件在哪里（**文件目录**）、**类型**（文本文件还是二进制文件）、当前是**读还是写**、**读写的位置**、**缓冲区**等信息，这些信息都被记录在一个称为 **FILE 结构类型** 的**结构**中。

FILE 结构类型：是一种专门用于记录正在读/写的文件的信息的结构类型。其中**文件的信息包括**：**文件名、文件目录、当前读写的位置指示，缓冲区指针、缓冲区大小、读写标志（读还是写）**等。

如：Visual C++ 6.0中的FILE结构类型定义如下

```
typedef struct _iobuf {  
    char *_ptr;    /*当前读写位置指针，指向缓冲区中的字节*/  
    int _cnt;      /*缓冲区的字符计数器，区分空或满*/  
    char *_base;   /*指向缓冲区基地址指针*/  
    int _flag;     /*文件状态标志，只读、读写、追加等*/  
    int _file;     /*文件描述符*/  
    int _charbuf;  /*非缓冲回送字符*/  
    int _bufsiz;   /*缓冲区大小*/  
    char *_tmpfname; /*临时文件描述符*/  
} FILE;
```

注：不同的C编译器关于FILE结构类型的定义会有所差异，但记录的信息基本是一致的。

C程序运行的时候，将记录所有**正在读写的文件**信息。这些文件的信息被记录在**系统创建和维护**的一个**FILE类型的结构数组**中。
该结构数组的定义类似于：

```
FILE _io[FOPEN_MAX];
```

其中，**FOPEN_MAX** 是系统预定好的一个整型符号常量，规定了结构数组的大小，也规定了系统允许程序同时打开的最大文件数。C语言标准规定流的**数目不得小于8**，具体由编译器制定，如VC中定义：**#define FOPEN_MAX 20**。

因此，FILE结构对程序而言是一种**稀缺资源**：**FILE 结构数组**只有 **FOPEN_MAX** 个元素，一个C程序中能同时读写的文件数量最多只有 FOPEN_MAX 个。

(1) 打开文件:

打开文件用 C 标准 I/O 函数 **fopen**:

FILE * fopen(char * **filename**, char * **mode**);

功能: 按 **mode** 指定的方式打开由 **filename** 指定的文件。

如: **FILE * fp = fopen**("abc.dat", "r");

fopen的操作:

在 **FILE 结构数组 _iob** 中申请一个“**空闲**”元素, 用于记载现在要打开的文件的信息, 并完成创建缓冲区、设置读写位置指针等一系列初始化工作, 从而**建立 “流” 与外存中指定文件的连接, 为文件的读写做好准备。**

fopen 函数返回值是其申请到的 **_iob** 结构数组中一个元素的地址，类型是 **FILE ***，并赋给一个 **FILE *** 指针变量。

如：

```
FILE * fp = fopen("abc.dat", "r");
```

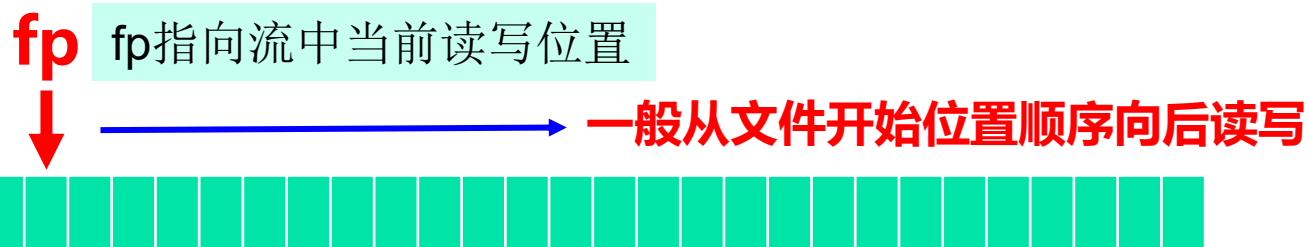
这里，fp 称为**文件指针**，代表一个文件在内存中的存在。以后**对文件的读、写都是基于文件指针进行的**。

注：如果文件打开过多（如超过20个），FILE结构数组中没有空闲元素或发生其它异常（如文件不存在），会导致 **fopen 执行失败**，此时 fopen 返回 NULL。

- ◆ 可通过 **if(fp == NULL)** 判断文件是否打开成功。

对打开失败的文件（空文件指针）进行读/写，系统会报错。

- ◆ 文件指针相当于文件指示器，**简单理解**就是指示着**文件当前的读写位置**（实际上由是 FILE 结构中的数据成员 **_ptr** 指示）。
- ◆ 文件的读写都会造成**文件指针向后移动** —— 即原则上**文件是从前往后 “顺序” 读写**；



- ◆ 如果想**前移文件指针**，需要用特定的函数来设置文件指针的位置，以实现前移，进而可以实现**文件的 “随机” 读写**。

□ 文件的打开模式 (**mode**)

```
FILE * fopen(char * filename, char * mode);
```

mode 是字符串，由 **r**、**w**、**a**、**t**、**b**、**+** 六个字符组合而成，其中：

- ◆ **r** 表示以**只读**方式打开的文件，不能写和更新。
- ◆ **w** 表示以**只写**方式打开文件。如果打开时文件还不存在，则创建新文件，如果已存在，则清空现有文件，从头开始写。
- ◆ **a** 表示打开文件后，把新写的内容添加在文件已有内容的后面。
- ◆ **+** 表示更新，打开的文件可读可写。
- ◆ **t** 表示以**文本格式**打开文件，读写的内容是**字符型数据**。
- ◆ **b** 表示以**二进制格式**打开文件，读写的内容是“**值型**”字节。

六个字符组合之后，文件的打开方式主要有以下几种：

- **r 或rt**: 以只读方式打开文本文件, 从头读;
- **w或wt**: 打开或创建文本文件、只写, **对已经存在的文件, 清除原有内容, 从头覆写**;
- **a 或at**: 添加, 打开或创建文本文件, **对已经存在的文件保留现有内容, 然后在尾部进行写**;
- **rb**: 以只读方式打开二进制文件, 从头读;
- **wb**: 打开或创建二进制文件、只写, 对已经存在的文件, 清除原有内容, **从头覆写**;
- **ab**: 添加, 打开或创建二进制文件, 在尾部进行写;
- **r+ 或rt+ 或r+t**: 打开文本文件, 从头读, 也可更新, 如果写, 将覆盖文件指针当前所指字节的内容;
- **w+或wt+ 或w+t**: 打开或创建文本文件, 对已经存在的文件, **清除原有内容**; 从头写, 并可更新;
- **a+ 或at+ 或a+t**: 添加, 打开或创建文本文件, **在尾部写**, 可更新;
- **r+b 或 rb+**: 打开二进制文件, 从头读, 也可更新, 如果写, 将覆盖文件指针当前所指字节的内容;
- **w+b或 wb+**: 打开或创建二进制文件, 对已存在文件, 清原有内容; 从头写, 并可更新;
- **a+b 或 ab+**: 添加, 打开或创建二进制文件, 在尾部写, 可更新。

(2) **fclose**函数：关闭文件

```
int fclose (FILE * fp);
```

- ◆ 功能：如果文件缓冲区中有尚未存盘的数据，将全部强制性的存盘；然后释放打开文件时系统分配的输入输出缓冲区，取消FILE指针与文件之间的映射关系，释放对FILE结构数组元素的占用。之后就不能再用该文件指针访问文件了，除非重新打开。
- ◆ 如果文件正常关闭，**fclose**函数返回0，否则返回非0。

注：打开的文件，使用完后一定要关闭。否则文件一直处于打开状态，即FILE结构数组元素一直被占用，造成资源浪费。

例10.3 文件以**文本文件格式**创建，以**二进制文件格式**读取。

```
#include "stdio.h "
```

```
int main(void)
```

```
{  char a[20];
```

```
    unsigned char u,i=0;
```

```
    FILE *fin,*fout;          /*定义文件指针，fin用于读文件，fout用于写文件*/
```

```
    fout=fopen("c:\\abc.dat","w+");  /*以文本写+更新的方式创建c盘根目录下的文本文件abc.dat*/
```

```
    while(gets(a)!=NULL)              /*输入一行字符串，当输入不是ctrl+z循环*/  
        fprintf(fout,"%s",a);
```

```
    fclose(fout);                  /*关闭fout所指的文件*/
```

```
    fin=fopen( "c:\\abc.dat" , "r+b" ); /*按二进制文件读+更新方式打开abc.dat文件*/
```

```
    while(!feof(fin)){              /*若没有到文件尾*/
```

```
        fread(&u,sizeof(unsigned char),1,fin);  /*从文件中读1个字节到u中*/
```

```
        if(!feof(fin)){              /*再次判断是否到文件尾*/
```

```
            printf("%04x  %x\t",i++,u);          /*输出顺序号，u中字符*/
```

```
            if(i%4==0)                  /*格式控制，每行输出4个字符信息*/
```

```
                printf("\n"); } }
```

```
    printf("\n");
```

```
    fclose(fin);                  /*关闭fin所指文件*/
```

```
    return 0;
```

```
}
```

注：同一个文件，既可以以文本方式打开，也可以用二进制方式打开，这主要**在于程序如何用这个文件**，与文件本身的内容没有绝对的关联。

□ 文件的操作步骤

对任何文件的操作，基本遵循以下三步：

第一步，**打开文件**

建立文件指针（或文件句柄）与文件间的联系；

第二步，**读/写**

对文件进行**读、写操作**；

第三步，**关闭文件**

取消文件指针（或文件句柄）与文件的联系。

10.3 流式文件的顺序输入与输出

C语言对**流式文件**的读写方式有两种：**顺序读写**和**随机读写**。

本节介绍流式文件的**顺序读写**：

1、基本的文件读写函数

以前基于键盘和屏幕的“**标准**”**输入输出**，默认是从键盘接收输入，向屏幕输出信息。而文件读写时，要指明**从哪个文件读数据**或**向哪个文件写数据**。

(1) 从文件输入、输出一个字符的函数：

```
int getc(FILE * fp)    #define getc(fp)   fgetc(fp)
```

```
int putc(int c, FILE * fp)
```

```
int fgetc(FILE * fin);
```

```
int fputc(int c, FILE * fout);
```

(2) 从文件输入、输出字符串的函数：

```
char * fgets(char *s, int n, FILE *fp);
```

```
int fputs(const char *s, FILE *fp);
```

(3) 从文件格式化地读写

```
int fprintf(FILE * fp, const char *format, ...);
```

```
int fscanf(FILE * fp, const char *format, ...);
```

- ◆ 每个函数都带有一个**文件指针参数**，用于说明从哪个文件读或向哪个文件写。
- ◆ 这些函数的名称和功能与“标准”输入输出相应的函数的名称和功能基本一致，可对照理解。

2、三个标准文件指针和标准输入输出函数

(1) 三个标准文件指针: **stdin**、**stdout**、**stderr**

C语言预定义了三个标准流式文件指针，简称**标准文件**。

```
#define stdin    (&_iob[0])
```

```
#define stdout  (&_iob[1])
```

```
#define stderr  (&_iob[2])
```

对应 **FILE** 结构数
组的前三个元素

- **stdin**: 标准输入文件，对应标准输入设备，通常是键盘
- **stdout**: 标准输出文件，对应标准输出设备，通常是显示器
- **stderr**: 标准出错文件，对应标准出错信息输出设备，始终是显示器
- 这三个标准文件是系统默认自动打开的，可以直接引用

(2) 标准输入输出函数

- ◆ 标准输入函数，如 **getchar**、**gets**、**scanf**，就是**系统默认从标准输入文件中读数据**，文件指针缺省，但对应着**stdin**；
- ◆ 标准输出函数，如 **putchar**、**puts**、**printf**，就是**系统默认向标准输出文件写数据**，文件指针缺省，但对应着**stdout**。

int **getchar**(void); ➡ **#define getchar() getc(stdin)**

int **putchar**(char ch); ➡ **#define putchar(c) putc((c), stdout)**

系统定义的宏函数

int **fgetchar**(void); 等价于 **fgetc(stdin)** 专指从**标准输入设备**中读一个字符

int **fputchar**(int c); 等价于 **fputc(c, stdout)** 专指向**标准输出设备**中写一个字符


```
void copy_TERM (const char * filename)
```

```
{ FILE *fp;  char ch;
```

```
  if((fp = fopen(filename, "w")) == NULL) {
```

```
    printf("can't open the file!");    exit(-1);
```

```
}
```

```
while((ch = fgetc(stdin)) != EOF) /*从键盘读取字符并赋给ch, 可用getc*/
```

```
    fputc(ch, fp);                /*向fp指向的文件写一个字符 */
```

```
fclose(fp);
```

```
}
```

```
void type(const char * filename)
```

```
{  char ch; FILE *fp;
```

```
  if((fp = fopen(filename, "r")) == NULL) { printf("can't open the file!"); exit(-1); }
```

```
  while((ch = fgetc(fp)) != EOF) fputc(ch, stdout); /*从fp指向的文件读一个字符输出到显示器*/
```

```
  fclose(fp);
```

```
}
```

一般, **fopen**后应判断文件是否成功打开, 打开失败时fp==NULL, 不能对文件进行操作, 否则出错。

以前用过的一些标准输入输出函数：

`char * gets(char *s);` 专指从标准输入流**stdin**中读一行字符。

`int puts(const char *s);` 专指向标准输出流**stdout**输出一行字符。

`scanf(...);` 专指从标准输入文件**stdin**输入

`printf(...);` 专指向标准输出文件**stdout**输出

3、基于缓冲区的格式输入输出函数

内存中一块连续的字节空间，通常称为**缓冲区**。

缓冲区通常以**一维字节空间**（**一维字符数组**或**malloc申请的字节空间**）的形式定义，可以向缓冲区输出数据或从缓冲区输入数据。C 语言提供了一组**基于缓冲区的格式化输入输出函数**：

```
int  sprintf(char *buffer, const char *format, ...);
```

```
int  sscanf(char *buffer, const char *format, ...);
```

- ◆ **sprintf** 向 **buffer** 所指示的缓冲区中写入格式串内容，输出的格式和 **printf** 完全一样，只是改为向 **buffer** 输出。
- ◆ **sscanf** 从 **buffer** 所指示的缓冲区中读入格式数据，输入的方式和 **scanf** 完全一样，只是改为从 **buffer** 输入。

例10.10 sprintf、sscanf函数的使用举例。

```
int main(void)
{
    char str[80], a[20] = "data", b[30];
    int x=123;
    sprintf(str, "%s %s %d", "input", a, x);
    printf("%s\n", str);
    sscanf(str, "%s %s %d", b, a, &x);
    printf("%s\t%s\t%d\n", b, a, x);
    return 0;
}
```

特殊的用途：

构造包含**动态内容**的字符串。

程序运行结果

input data 123

input data 123

注：使用sprintf、sscanf函数进行输入输出，特别是输出，要构造一个**足够大的数组**，使之能够容纳输入输出的数据。

4、文件的直接输入输出

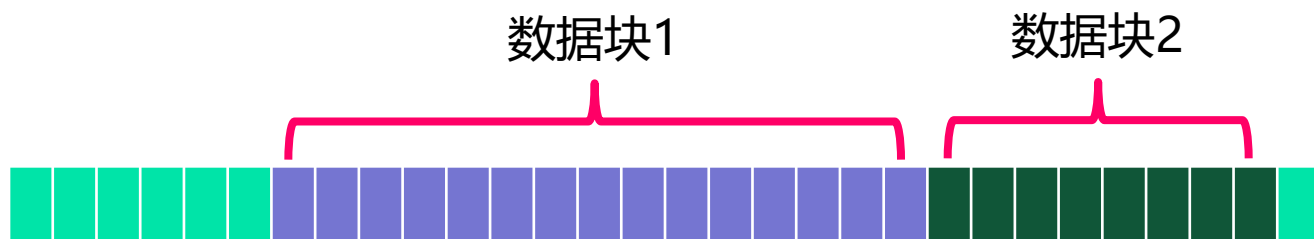
前面的文件输入输出**本质上都是对文本文件**进行操作，如：

- ◆ printf、sprintf、fprintf：以格式字符串的方式输出数据，结果是向屏幕、缓冲区、文件中输出一个字符串的内容；
- ◆ scanf、sscanf、fscanf：接收从键盘、缓冲区、文件中输入一个字符序列，然后按照格式串中格式转换符的要求转换成其它类型的数据。

而下面的**文件的直接输入输出**是指从**文件指针的当前位置**开始**一次写入或读出由若干连续字节组成的数据块**，而不用管其中的具体内容（即当作**字节流式数据**直接写入或读出）。

◆ 文件的直接输入输出

文件的直接输入输出是指从**文件指针的当前位置**开始**一次写入或读出由若干连续字节组成的数据块**，而不用管其中的具体内容（即当作**字节流式数据**直接写入或读出）。



- C语言的标准文件直接输入输出函数有两个：

fread 和 **fwrite**

fread和fwrite函数从输入文件读或向输出文件写一个**数据块**，数据块之间没有分隔符分隔。

(1) fread函数

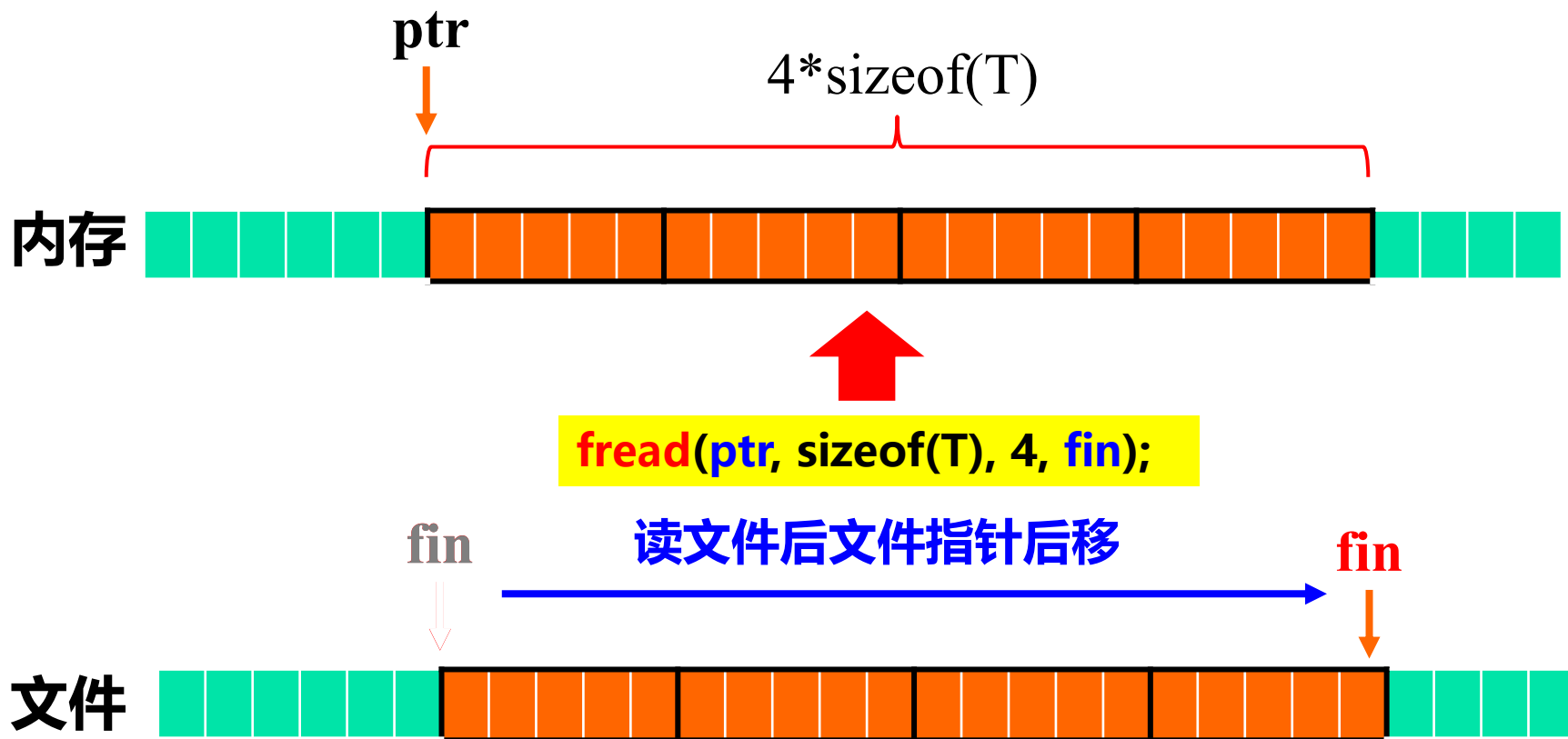
size_t fread(void *ptr, size_t size, size_t n, FILE * fin);

功能：从文件指针 **fin** 处读入 **size*n 个字节**存入 **ptr** 所指的**内存缓冲区**中，这些字节可以分成 **n 个 size 大小的记录**。

参数：

- **ptr**：指向**内存缓冲区的指针**，缓冲区用于接收读入的数据
- **size**：一个记录的**字节长度**
- **n**：一次读取的**记录个数**， $n \geq 1$
读入的数据块字节总量 = size * n 个字节
- **fin**：输入文件的**文件指针**。

返回值：**实际读取的记录数**，至多读取n个记录。



从文件指针 `fin` 指示的输入文件的当前位置处一次性“整块”读取连续的 $4 * \text{sizeof}(T)$ 个字节内容，存到内存中由 `ptr` 指示起始位置处开始的 $4 * \text{sizeof}(T)$ 个字节中。文件指针 `fin` 在读文件的过程中随之后移。

(2) fwrite函数

size_t fwrite(const void ***ptr**, size_t **size**, size_t **n**, **FILE * fout**);

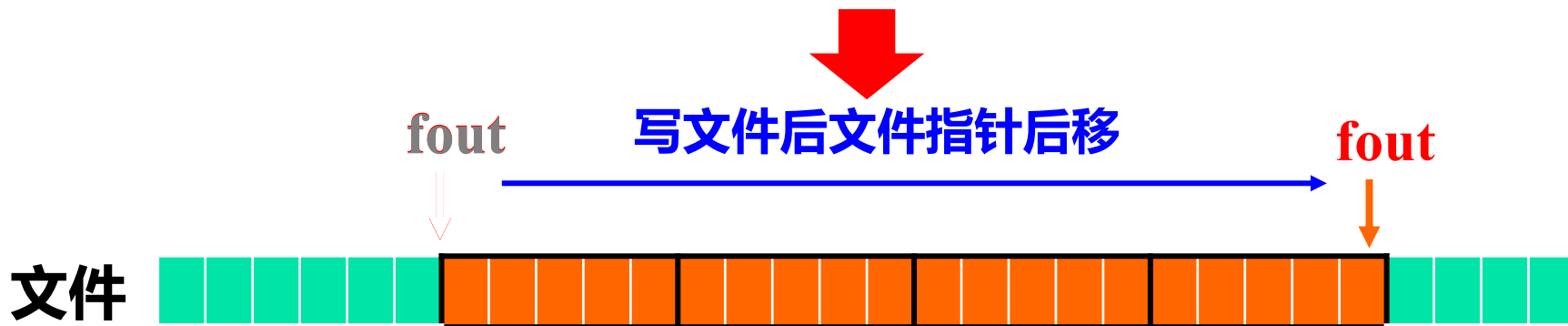
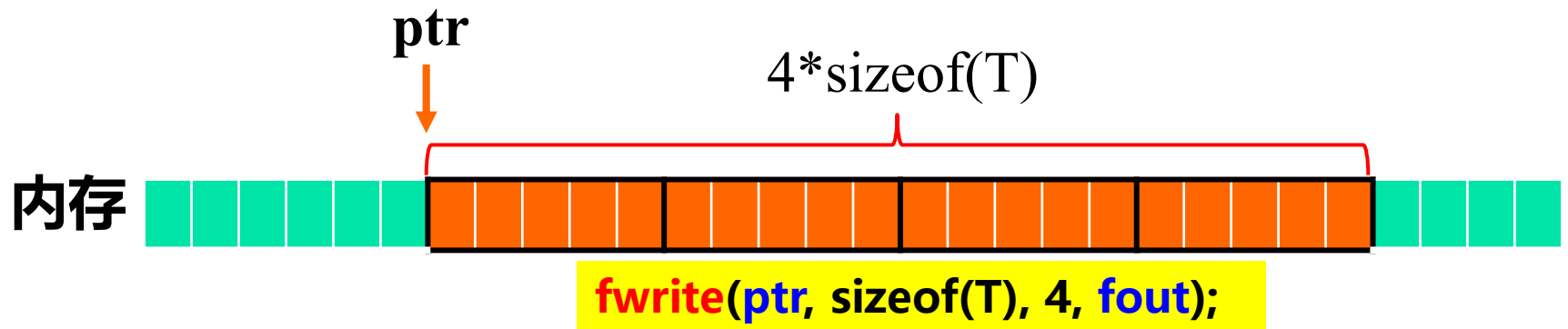
功能： 把从ptr指向的内存缓冲区开始处存储的n条大小为size的记录 (共**n*size**个字节) 一次性写到fout指向的输出文件中。

参数： **ptr**、**size**、**n**的含义均同fread;

写入的数据块大小 = **size** * **n** 个字节

fout：指向输出文件的**文件指针**。

返回值： 输出的记录个数；操作出错时，返回值小于n。



从 ptr 指示的内存起始位置开始，后面的连续 $4 * \text{sizeof}(T)$ 个字节一次性“整块”复制到文件指针 fout 指示的输出文件中，fout 随之后移。

注： **fread**和**fwrite**以**字节流的方式**存取文件数据（即流式数据，它们不关心数据块内部的逻辑结构和其中所含数据的类型等，仅作为（二进制）字节型数据读入或写出），所以针对的一般都是**二进制文件**。

(3) **feof**函数

```
int feof(FILE * fp);
```

功能：测试文件指针是否已经到文件结尾

如果处于文件尾，返回**非0值**；否则返回**0**。

一般读文件后调用feof(fp)**来判断文件是否读完。**

例10.12 大学生基本信息类型 **struct stu_info** 如下，生成大学生基本信息文件并读出。

大学生基本信息结构类型：

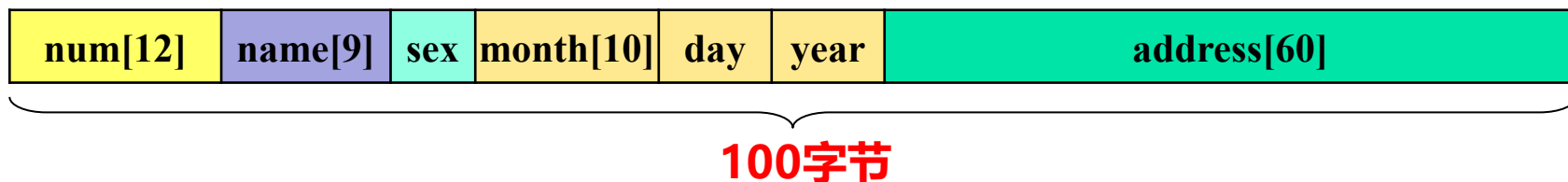
```
struct stu_info {  
    char num[12];          /* 学号 */  
    char name[9];          /* 姓名 */  
    char sex;              /* 性别 */  
    struct date birthday; /* 出生日期 */  
    char address[60];      /* 家庭住址 */  
};
```

struct date {
 char month[10];
 int day;
 int year;
};

嵌套结构

每个学生一条 **struct stu_info** 记录。

每条记录的大小 = **sizeof(struct stu_info)** = **100字节**



```
#include "stdio.h"
```

```
int main(void)
```

```
{ int x=1; /*x是循环控制标识, x==-1结束循环*/
```

```
    struct date d;
```

```
    struct stu_info s,*p=&s;
```

```
    FILE *fin,*fout;
```

```
    if((fout = fopen("c:\\stu_info_table.dat","wb")) == NULL) exit(-1);
```

```
    while(x!=-1){
```

输入一个学生记录

```
        printf("输入出生日期, 格式: 月日年\n");
```

```
        scanf("%s%d%d",d.month,&d.day,&d.year); getchar(); s.birthday=d;
```

```
        printf("输入学号、姓名、家庭住址、联系电话、电子邮箱\n");
```

```
        scanf("%s%s%s%s%s",s.num,s.name,s.address); getchar();
```

```
        printf("输入性别\n"); scanf("%c",&b>s.sex); getchar();
```

```
        fwrite(&s, sizeof(struct stu_info), 1, fout); 一条学生记录作为一个数据块
```

```
        printf("继续输入吗? -1结束\n"); scanf("%d",&x); getchar(); /*x==-1时结束*/
```

```
    }
```

```
    fclose(fout);
```

例10.12-continue

```
if((fin = fopen("c:\\stu_info_table.dat","rb")) == NULL) exit(-1);  
while(!feof(fin)) {  
    fread(&s, sizeof(struct stu_info), 1, fin);  
    if(!feof(fin)){  
        printf("学号=%s\t姓名=%s\t性别=%c\t", s.num, s.name, s.sex);  
        printf("出生日期=%4d-%s-%d\n",s.birthday.year, s.birthday.month, s.birthday.day);  
        printf("家庭住址=%s\n", s.address);  
    }  
}  
fclose(fin);  
}
```

一次读入一个学生记录的数据块

判断是否读到文件结尾了

fwrite将一个结构的内容（所有字节）以“**原始字节形态**”写入文件；而**fread**将其“**原样**”读出，覆盖结构变量中各个成员的字节空间后，就可以“**原样**”解析其中的内容。

num[12]	name[9]	sex	month[10]	day	year	address[60]
---------	---------	-----	-----------	-----	------	-------------

5、文件的重定向

- ▣ **文件重定向**：是指使一个**已存在**的原本指向A文件的文件指针**改为指向另外的文件B**。

- ▣ **freopen** 函数实现文件指针重定向：

FILE * freopen (const char * **filename**, const char ***mode**, **FILE * fp**);

功能：freopen 函数将首先关闭文件指针 fp 当前所指文件，然后按照 mode 指定的方式打开 filename 指定的文件，并使 fp 重新指向该新打开的文件。

- 若重定向成功，**freopen** 函数使 **fp** 指向新打开的文件，并返回 **fp** 的当前指针值；如果打开 filename 失败，返回 NULL。

*10.4 流式文件的随机输入输出

文件的**随机输入输出**是指对文件的读写可以从文件内**指定的任意位置**进行，而不必每次从头顺序开始。

随机输入输出需要**对文件指针进行定位**，由 C 语言提供的**标准文件定位函数**的支持。

- C 提供了五个文件指针定位函数，在stdio.h头文件中声明。

```
int fseek(FILE *stream, long offset, int whence);
```

```
long ftell(FILE *stream);
```

```
int fgetpos(FILE *stream, fpos_t *pos);
```

```
int fsetpos(FILE *stream, const fpos_t *pos);
```

```
void rewind(FILE *stream);
```


****10.6 输入输出的底层接口**

- ▣ 与流式文件使用 C 语言定义的基于缓冲的函数不同，**输入输出的底层接口**中使用的函数不是由 C 语言定义，而是**通过 UNIX、DOS 及 Windows 等操作系统提供的功能调用实现**（也称为**接口**）。
- ▣ 输入输出的底层接口不使用 **FILE** 文件指针，而是使用**整型文件说明符**（也称为**文件句柄**）访问文件。

10.6.1 文件顺序输入输出的“底层接口”函数

文件顺序输入输出的底层接口包括文件的创建、打开、读写、关闭等操作，由 **creat**、**open**、**read**、**write**、**close** 等一组函数提供。

- ◆ 这些函数的原型声明在 **io.h** 头文件中，使用时应该包含

#include "io.h"

▣ `int creat(const char *path, int amode);`

功能：根据 **path** 指向的路径和文件名创建一个新文件或准备重写一个已存在的文件。

返回值：如果打开成功，返回一个大于或等于 0 的**文件说明符**；
否则返回 -1。

打开方式：由 **stat.h** 中定义的如下符号常量规定：

```
#define S_IREAD    0x0100    /* 只读方式 */
```

```
#define S_IWRITE   0x0080    /*只写方式*/
```

例如，以读写方式创建 C 盘根目录下的 **low_io.dat** 文件。

```
int handle;
```

```
if((handle = creat("c:\\low_io.dat",S_IREAD|S_IWRITE))== -1)
```

```
    exit(-1);
```

```
else printf("%d\\n",fp);
```

□ int **open**(const char ***path**, int **access**, unsigned **mode**);

功能：根据 **path** 指向的路径和文件名打开一个文件，并用**参数 access** 设置文件的**访问控制属性**，用**参数 mode** 设置文件的**读写方式**。

返回值：如果打开成功，返回**文件说明符**；打开失败返回-1。

访问属性：在 **fcntl.h** 中定义。例如：

```
#define O_RDONLY    1    /* 打开文件只读 */
```

```
#define O_WRONLY    2    /* 打开文件只写 */
```

```
#define O_RDWR      4    /* 打开文件读写 */
```

- 这三个常量是基本访问控制属性。它可以进一步与下面的常量按**位或**形成进一步的修饰，其他的修饰常量见fcntl.h头文件。

```
#define O_CREAT      0x0100    /* create and open file */
```

```
#define O_TRUNC      0x0200    /* open with truncation */
```

```
#define O_EXCL       0x0400    /* exclusive open */
```

如：int **handle**;

```
handle = open("C:\\\\low_io1.dat", O_WRONLY|O_CREAT, S_IREAD|S_IWRITE);
```

它以读写方式打开c盘根目录下的low_io.dat文件，允许写，允许创建。

□ `int read(int handle, void *buf, unsigned len);`

功能：从 **handle** 指示的文件中读 **len** 个字节到指针 **buf** 所指向的缓冲中去。

返回值：实际所读的字节数(在将要到达文件尾时，返回值可能小于 len)；

返回 0 表示到达文件尾；返回 -1 表示读操作出错。

□ `int write(int handle, void *buf, unsigned len);`

功能：从 **buf** 所指向的缓冲中取 **len** 个字节写到 **handle** 指示的文件中。

返回值：实际所写的字节数，返回 -1 表示读操作出错。

注：**read** 和 **write** 函数是**以字节为单位**对打开或创建的任意格式文件进行读写，所读写的文件无文本或二进制之分。

➤ 其读写的字节所代表的数据或编码含义与读写操作无关，含义由字节使用者自行解释

▣ `int close(int handle);`

- 功能：关闭 `handle` 指示的文件。
- 返回值：如果成功关闭文件，返回 0，否则返回 -1。

10.6.2 文件的随机输入输出底层接口函数

- 当用 **creat** 函数创建新文件或用 **open** 函数打开已有文件进行读写时，操作系统会对被读写的文件建立**文件位置指示器**，并且随着读写操作的进行，**文件位置指示器的值会顺序增加**。
- 上面讨论的是文件的**顺序读写**，而如果希望从文件任意指定位置进行读写，就需要**使用底层接口函数lseek进行读写定位，用tell函数获取文件读写的当前位置**。
- 底层接口提供的对文件进行随机输入输出的函数主要有：
lseek、tell、eof

▣ long **lseek**(int **handle**, long **offset**, int **origin**);

功能：将 **handle** 说明的文件的读写位置移动到以参数 **origin** 为基准，偏移量为 **offset** 的位置处。

◆ origin 的取值可以是0、1、2

(对应符号常量 SEEK_SET、SEEK_CUR及SEEK_END)

返回值：如果调用成功，lseek函数返回偏移量（offset的值）；
否则意味着调用失败。

▣ long **tell**(int **handle**);

功能：返回由 handle 说明的文件的当前读写位置值，该值为从文件开始处到当前位置间的字节数。

返回值：如果调用失败，tell 函数返回 -1。

本章小结

文件的有关概念： 文本文件/二进制文件、顺序文件/随机文件，C程序输入输出的实现方法、FILE结构类型、FILE指针、标准流式文件。

流式文件的顺序输入输出： 文件的打开与关闭、文件的重定向、基于字符和基于字符串的文件读写、文件的格式读写、文件的直接输入输出。

流式文件的随机输入输出： 文件定位函数，以随机读写方式编制填写班级成绩单文件的程序例子。

C语言的其他文件操作函数： 文件访问函数、文件操作函数、出错检测处理函数。

输入输出的底层接口： 与操作系统密切相关，又包括文件顺序输入输出和文件随机输入输出，通过例子介绍了实现文件I/O相关的接口函数的使用。

刷题题、练练手

<https://www.luogu.com/>

- 洛谷创建于 2013，至今已有百万用户，致力于为编程爱好者（oiers/acmers）提供清爽、快捷的编程体验。它不仅仅是一个在线测题系统，更拥有强大的社区、在线学习功能。同时，许多教程内容都是由五湖四海的用户提供的，保证了内容的广泛性。无论是初学算法竞赛的蒟蒻，还是久经沙场的神犇，均可从洛谷获益，也可以帮助他人，共同进步。

<http://leetcode.com/>

- LeetCode 是一个很受欢迎的在线判题系统，可以帮助挑战者为**面试**做好技术准备。挑战者可以用 9 种编程语言直接在线完成挑战。

<http://www.topcoder.com/>

- 是一个**程序设计比赛**的网站，有近百万程序员支持，每个星期都有两次网上在线比赛，根据比赛的结果对参赛者进行新的排名。

其它：POJ、HDUOJ，牛客网，Codeforces等

参与编程竞赛活动

1. ACM ICPC

ACM 国际大学生程序设计赛，国际上最为认可的程序设计大赛。

2. CCF CSP

CCF计算机软件能力认证，由中国计算机学会主办，国内最具权威的程序设计认证考试。

3. CCF CCSP

为CSP优秀成绩者举办的竞赛

4. CCCC（中国高校计算机大赛-团体程序设计天梯赛）

教育部教指委主办的程序设计赛

5. 蓝桥杯

工信部主办的比赛，分省赛和国赛，省赛简单，国赛水平还可以。